Vrije Universiteit Amsterdam

ML Project X_400154

Evaluating Regression Models for Predicting Workplace Productivity in AI-Augmented Environments

Authors:

Zoe Lea Petroianu	(2809310)
Aida Marija Bielinskaite	(2804099)
George Dedempilis	(2803764)

Supervised by Teaching Assistant Jakub Kuwalek

March 19, 2026

Contents

1 Introduction **3**
1.1 Related Work . . . 3
1.2 Research Questions . . . 3

2 Data and Pre-processing **3**
2.1 The Dataset . . . 3
2.2 Cleaning the Dataset . . . 4

3 Methodology **4**
3.1 Outline . . . 4
3.2 Models . . . 4
3.2.1 Baseline: Mean Predictor . . . 4
3.2.2 Baseline: Linear Regression . . . 5
3.2.3 Ridge L2 Regression . . . 5
3.2.4 Decision Tree . . . 5
3.2.5 XGBoost . . . 5
3.2.6 Neural Network . . . 5
3.3 Hyperparameter Selection and Cross Validation . . . 6

4 Results **6**
4.1 Cross-validation performance . . . 6
4.2 Statistical Significance Test . . . 6
4.3 Model Comparison (Error Bars) . . . 7
4.4 Residual Analysis results . . . 7
4.5 Learning Curves . . . 7
4.6 Ridge Coefficients . . . 7
4.7 Final Model Evaluation (L2 Ridge) . . . 7

5 Discussion **9**
5.1 Cross-validation performance . . . 9
5.2 Statistical significance test . . . 9
5.3 Model Comparison (Error Bars) . . . 9
5.4 Residual Analysis . . . 9
5.5 Learning curves . . . 9
5.6 Ridge Coefficients . . . 9
5.7 Final model evaluation (L2 Ridge) . . . 9

6 Limitations and Future Work **9**

7 Conclusion **10**

Abstract

This paper investigates how well machine learning models can predict worker productivity in AI-augmented work environments. Using a dataset of 4,500 employees with 14 features describing their characteristics and AI-usage variables, we compared two baselines (mean predictor and ordinary linear regression) with Ridge L2 Regression, decision trees, XGBoost, and neural networks. Models were evaluated with 5-fold cross-validation and optimized using simulated annealing. Ridge L2 Regression performed best ($R^2 = 0.8788$, MAE = 3.9428, RMSE = 4.9105 on the test set). Task automation, experience, and focus hours were the strongest positive predictors of productivity, while error rate and burnout risk were the strongest negative predictors. These results suggest that productivity in AI-augmented work environments can be predicted with high accuracy.

1 Introduction

Artificial intelligence tools are becoming increasingly common in the modern workplace. From code assistants to writing aids, employees now have access to AI systems that can automate routine tasks and support decision-making. As more and more organizations invest in these technologies, a question arises: how do AI tools affect worker productivity, and can we predict productivity outcomes based on information about workers and their AI usage?

While recent research has shown that AI assistance can influence worker performance, most of this work focuses on measuring the effects of AI in specific task settings. In contrast, our project takes a predictive modeling perspective. We investigate whether productivity scores can be predicted from worker-related features and AI-usage variables in a structured workplace dataset. Specifically, we compare two baselines - a mean predictor and ordinary linear regression - with several models of varying complexity: Ridge L2 regression, decision trees, XGBoost, and a neural network. Our goal is not only to identify which model performs best, but also to understand which features are most predictive and whether additional model complexity yields meaningful improvements.

1.1 Related Work

Research on job performance long predates the recent interest in AI tools. In organizational psychology, Judge et al. [10] conducted a meta-analysis of the relationship between job satisfaction and job performance. Across 312 samples, they estimated a mean true correlation of 0.30 between the two variables, though they noted that the causal relationship between job satisfaction and job performance remains unresolved. Their work highlights the difficulty of explaining performance from workplace factors.

More recently, researchers have examined how AI tools specifically affect productivity. Brynjolfsson et al. [1] studied the phased introduction of a generative AI assistant among 5,172 customer support agents. They found that access to the tool increased productivity by 15% on average, with the largest improvement observed among less experienced and lower-skilled workers. Dell'Acqua et al. [3] conducted a preregistered field experiment with 758 management consultants and found that AI assistance improved performance on tasks within the AI's capabilities, but reduced accuracy on tasks outside the range of problems the AI could solve reliably.

Similar results appeared in other domains. In software development, Peng et al. [13] ran a controlled experiment and found that developers with access to GitHub Copilot completed coding tasks 55.8% faster than a control group. Noy and Zhang [12] observed comparable productivity gains in professional writing, showing that access to ChatGPT reduced average completion time by 40% and improved output quality by 18%. Together, these studies suggest that AI can meaningfully improve productivity, but the size of the effect varies across workers and task settings.

1.2 Research Questions

Our project is guided by one main research question:

RQ: How do different machine learning models - Ridge L2 Regression, Decision Trees, XGBoost, and Neural Networks - perform in predicting the productivity of workers who use AI tools in the workplace?

We also address two secondary questions:

SQ1: Which features are most predictive of productivity?

SQ2: How does hyperparameter selection affect model performance?

2 Data and Pre-processing

2.1 The Dataset

We use the *AI Workplace Productivity Dataset* published on Kaggle by Vishard Mehta [11]. The dataset describes 4,500 employees working in AI-augmented environments. We selected it because it contains a relatively large number of instances, provides a clear numeric target variable (`productivity_score`), and a mix of numerical and categorical features, making it well-suited to regression and model comparison.

Each observation corresponds to a single employee and includes 14 input features along with two target variables: a continuous productivity score (ranging from 20.5 to 100, representing overall worker performance) and a categorical burnout risk level (*Low*, *Medium*, *High*).

The input features can be grouped into three categories. The first category contains worker background variables: job role (a categorical variable with six levels: *Analyst*, *Designer*, *Developer*, *Manager*, *Marketer*, and *Writer*) and years of experience (ranging from 1 to 20 years).

The second category describes AI usage and work

patterns: hours per week spent using AI tools (0-30.4 hours), percentage of tasks automated (0-79.9%), manual work hours per week, learning time hours per week (0-43.3), meeting hours per week (0-30.2), collaboration hours per week (2-20), and focus hours per day (1-8).

The third category captures workplace conditions and well-being indicators: deadline pressure level (a categorical variable with three levels: *Low*, *Medium*, and *High*), error rate percentage (0.1-6.67%), task complexity score (an integer from 1 to 10), work-life balance score (1-10), and burnout risk score (3.22-10).

The dataset contains no missing values and no duplicate records. The job roles are unevenly distributed, with Developers representing the largest group (1,115 employees) and Writers the smallest (461 employees). The deadline pressure levels are approximately evenly distributed across Low (1,527), Medium (1,470), and High (1,503).

2.2 Cleaning the Dataset

Data cleaning involved several verification steps and transformations. First, sanity checks were performed to verify data integrity. Percentage variables (`tasks_automated` and `error_rate`) were confirmed to fall within the valid 0–100 range, while hour-based variables were checked to ensure that they were non-negative. In addition, years of experience were verified to be positive integers.

Next, the feature and target data, which were originally stored in separate files, were merged on the `Employee_ID` column. After merging, the `burnout_risk_level` variable in the target file was excluded from the analysis because it is a categorical encoding of the `burnout_risk_score` already present in the features, and including both would introduce redundancy.

Finally, categorical variables were transformed into numerical format using one-hot encoding. More specifically, the `job_role` variable was expanded into six binary columns (one for each role), and the `deadline_pressure_level` variable was expanded into three binary columns (*Low*, *Medium*, *High*).

3 Methodology

3.1 Outline

The goal of our project is to evaluate how well different machine learning models can predict the productivity score of workers who use AI tools in the workplace. The models considered range from simple linear methods to more flexible non-linear models. This allows us to observe how predictive performance changes as model complexity increases.

After preprocessing, the dataset is divided into a training set and a held-out test set using an 80/20 split. The training portion is used for model training and hyperparameter selection, while the test set is reserved exclusively for final evaluation. Numerical features are standardized using standard scaling, while categorical features (one-hot encoded columns) are passed through without transformation. This scaling is applied within the cross-validation pipeline so that the scaler is fit only on the training folds, not the validation fold.

Two simple baselines are first established to provide a point of comparison. More complex models are then used, including regularized linear regression (Ridge), decision trees, gradient boosted trees (XGBoost), and a feed-forward neural network.

Model performance is estimated using cross-validation on the training data. Hyperparameters are selected using simulated annealing. The objective function for simulated annealing is the mean R^2 across cross-validation folds. After hyperparameters are selected, the best model is retrained on the full training data and evaluated on the held-out test set.

Performance is evaluated using three standard regression metrics: mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination R^2:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{1}$$

[8]

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2}$$

[8]

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{3}$$

[9, eq. 3.17, pp. 69-70]

MAE was chosen because it gives a straightforward interpretation of the average prediction error in the same units as the target variable. RMSE was included because it penalizes larger errors more heavily, making it useful for detecting models that produce occasional large mispredictions. R^2 was used because it measures how much of the variance in the target variable is explained by the model [9], allowing direct comparison across models regardless of the scale of the data.

The equations presented in the following section use the standard formulations from James et al. (2021), though we opted for using w for model weights and α for the regularization parameter, in place of the β and λ notation used in the original text.

3.2 Models

This section provides an overview of the regression models used in this project, which all differ in flexibility.

3.2.1 Baseline: Mean Predictor

The first baseline predicts the mean productivity score observed in the training data for every instance. Let $y_1, y_2, \ldots, y_n$ denote the productivity scores in the training set. The baseline prediction is

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} y_i \tag{4}$$

for all instances.

Although this predictor ignores all feature information, it provides an important reference point. Any useful model should outperform the mean predictor by learning relationships between the input features and the productivity score.

3.2.2 Baseline: Linear Regression

Linear regression models the productivity score as a linear function of the input features. Given the feature vector $x = (x_1, x_2, \ldots, x_d)$, the model predicts

$$\hat{y} = w_0 + \sum_{j=1}^{d} w_j x_j \tag{5}$$

[9, p. 61]

where w_0 is the intercept and w_j are the model coefficients. The parameters are estimated by minimizing the sum of squared errors

$$\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{6}$$

[9, eq. 3.16, p. 69]

This model assumes that the relationship between each feature and the target is linear. While this assumption may be restrictive, linear regression is a strong baseline and is easy to interpret. Because it has no hyperparameters to tune, it is evaluated directly using cross-validation without simulated annealing.

3.2.3 Ridge L2 Regression

Ridge regression extends linear regression by adding L_2 regularization to the objective function. The model minimizes

$$\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{d} w_j^2 \tag{7}$$

[9, eq. 6.5, p. 237]

where α is the regularization parameter.

The regularization term discourages the model from relying too heavily on any single feature by penalizing large coefficient values. When α is close to zero, the model behaves almost identically to ordinary linear regression. As α increases, the coefficients are pushed closer to zero, which reduces overfitting at the cost of a less flexible model. The value of α is selected using simulated annealing, searching over the range 0.0001 to 100.

3.2.4 Decision Tree

Decision tree regression models the prediction function using a hierarchical partitioning of the feature space. The input space is recursively divided into regions using single-feature splits of the form

$$x_j < t$$

where x_j is a feature and t is a threshold.

Each split divides the data into two subsets. The tree is grown by selecting splits that reduce the variance of the target values within each region. Once the tree is constructed, predictions are made by assigning each observation to a leaf node and returning the average target value of the training examples in that leaf [9, ch. 8].

Tree models can capture non-linear relationships and interactions between features. However, deep trees can easily overfit the training data. For this reason, three parameters are selected using simulated annealing: maximum depth (2 to 30), minimum number of samples required to split a node (2 to 40), and minimum number of samples required at a leaf node (1 to 20).

3.2.5 XGBoost

XGBoost is an ensemble method that builds a sequence of decision trees where each tree corrects the errors of the previous tree. The final prediction is the sum of contributions from all trees in the ensemble

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i) \tag{8}$$

[2, eq. 1]

where K is the total number of trees and f_k is the prediction from the k-th tree.

The model has five hyperparameters that are selected using simulated annealing: the number of trees (50 to 1000), the learning rate (0.001 to 0.3), the maximum depth of individual trees (2 to 12), the row subsample ratio (0.5 to 1.0), which controls the fraction of training instances used per tree, and the column subsample ratio (0.5 to 1.0), which controls the fraction of features considered per tree.

3.2.6 Neural Network

The neural network is the most flexible model in the set. The network passes the input features through a series of hidden layers, where each layer applies a linear transformation followed by a non-linear activation function. For a single hidden layer, the output is computed as

$$h = ReLU(Wx + b) \tag{9}$$

[6, ch. 6, p. 168]

where W is the weight matrix, b is the bias vector, and $ReLU(z) = max(0, z)$ [9, eq. 10.5, p. 405]. ReLU was chosen as the activation function because it is computationally efficient and avoids the vanishing gradient problem that can slow training with other activations such as sigmoid [5]. The output of the final hidden layer is then passed through a linear output layer to produce the prediction. The network is trained using the Adam optimizer for up to 2000 iterations. It includes L_2 regularization on its weights to prevent overfitting and uses early stopping to halt training when validation performance no longer improves.

Four hyperparameters are selected using simulated annealing: the number of hidden layers (1 to 4), the num-

ber of neurons in each layer (16 to 128, 8 to 64, 8 to 32, 4 to 16 for layers one through four, respectively), the L_2 penalty strength α (0.000001 to 0.1), and the initial learning rate (0.0001 to 0.01).

3.3 Hyperparameter Selection and Cross Validation

The dataset is split into a training set (80%) and a held-out test set (20%) using a fixed random seed to ensure reproducibility. Before training, numerical features are standardized using standard scaling, while one-hot encoded categorical features (job role and deadline pressure level) are passed through without transformation. This scaling is applied within the cross-validation pipeline so that the scaler is fit only on the training folds, not on the validation fold, preventing data leakage.

Model performance is estimated using 5-fold cross-validation on the training set. The training data is divided into five equal subsets. In each iteration, one subset is held out for validation while the model is trained on the remaining four. This process is repeated five times so that every observation serves as validation data exactly once. Five folds were used because it is regarded as a good balance between the bias and variance of the performance estimate [9, sec. 5.1.4], and because using more folds would have significantly increased runtime, given that cross-validation is performed at every iteration of the simulated annealing search. Three metrics are computed on each fold: mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination (R^2). The mean and standard deviation of each metric across the five folds are reported to give both an estimate of performance and a measure of its variability.

Linear regression has no hyperparameters and is evaluated directly using cross-validation. The remaining four models each have hyperparameters that must be selected. This is done using simulated annealing, a global optimization method that can escape local minima during the search process. The objective function for simulated annealing is the mean R^2 across the five cross-validation folds, negated because the optimizer minimizes it, whereas we aim to maximize R^2. The optimizer is run for 50 iterations per model with a fixed random seed.

Table 1 summarizes the best hyperparameter configurations found during the simulated annealing search.

After simulated annealing identifies the best hyperparameters for each model, a final cross-validation is run with those hyperparameters to obtain the reported performance metrics. The model with the highest mean R^2 across the five folds is selected as the best model. This model is then retrained on the entire training set and evaluated on the held-out test set to produce an unbiased estimate of generalization performance.

Model	Best Hyperparameters
Ridge L2	$\alpha = 4.1785$
Decision Tree	max_depth = 10
	min_samples_split = 2
	min_samples_leaf = 11
XGBoost	n_estimators = 821
	learning_rate = 0.0168
	max_depth = 2
	subsample = 0.6117
	colsample_bytree = 0.7529
Neural Network	hidden_layer_sizes = (22, 12)
	$\alpha = 0.084$
	learning_rate_init = 0.0074

Table 1: Best hyperparameters selected during simulated annealing.

4 Results

4.1 Cross-validation performance

Model	MAE	RMSE	R^2
Mean predictor (Baseline)	11.25	14.11	-0.0004
Linear Regression (Baseline)	4.0139 ± 0.0774	5.0261 ± 0.1511	0.8770 ± 0.0064
Ridge	4.0133 ± 0.0756	5.0258 ± 0.1499	0.8771 ± 0.0064
Decision Tree	5.9994 ± 0.1011	7.5083 ± 0.1144	0.7255 ± 0.0066
XGBoost	4.1715 ± 0.0932	5.2355 ± 0.1440	0.8665 ± 0.0049
Neural Network	4.0657 ± 0.1095	5.1160 ± 0.1876	0.8724 ± 0.0086

Table 2: Cross-validation performance results.

As shown in Table 2, the cross-validation performance of the models was evaluated using three common regression metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score [9, ch. 3].

Linear Regression and Ridge Regression achieve the best performance with R^2 scores of approximately 0.877 and the lowest error values (MAE = 4.01, RMSE = 5.02). Ridge Regression achieves a slightly higher R^2 score (0.8771) compared to Linear Regression (0.8770), which is why it is selected as the best-performing model.

The Neural Network and XGBoost models show slightly lower performance with marginally higher prediction errors. In contrast, the Decision Tree model performs significantly worse, with a lower R^2 score (approximately 0.73) and higher error values. Additionally, hyperparameter optimization was performed using a simulated annealing search procedure during the cross-validation process.

Note: The mean predictor is included only as a reference baseline for comparison and was not part of the cross-validation procedure.

4.2 Statistical Significance Test

Model Comparison	t-statistic	p-value	Significant ($\alpha = 0.05$)
Linear Regression vs Ridge L2	0.8389	0.4487	NO
Linear Regression vs Decision Tree	67.8219	< 0.001	YES
Linear Regression vs XGBoost	8.1840	0.0012	YES
Linear Regression vs Neural Network	4.6807	0.0094	YES
Ridge L2 vs Decision Tree	69.4031	< 0.001	YES
Ridge L2 vs Neural Network	-0.7831	0.4773	NO

Table 3: Statistical significance test results.

To evaluate whether the performance differences between models are statistically significant, paired t-tests were conducted on the cross-validation scores across folds, following a common evaluation approach in machine learning [4]. Table 3 reports a subset of the pairwise comparisons between models. The results indicate that Linear Regression and Ridge Regression are not significantly different ($p = 0.4487$), while comparisons involving Decision Trees, XGBoost, and the Neural Network show statistically significant differences ($p < 0.05$).

4.3 Model Comparison (Error Bars)

The bar charts in Figure 1 show the mean performance and standard deviation for three regression metrics: R^2, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). Linear Regression and Ridge Regression achieve the highest R^2 scores (0.877) and the lowest prediction errors (MAE = 4.01, RMSE = 5.02). The Decision Tree performs significantly worse, with a lower R^2 score (approximately 0.73) and higher error values. XGBoost and the Neural Network achieve intermediate performance with R^2 values between approximately 0.867 and 0.872. Error bars represent the standard deviation across cross-validation folds, following the cross-validation framework described in [9, ch. 5].

4.4 Residual Analysis results

- Residual mean: 0.0099
- Residual standard deviation: 4.91

To further evaluate the reliability of the final model, a residual analysis was performed on the test set. Residuals represent the difference between the actual values and the predicted values. The residual statistics show a mean of approximately 0.0099 and a standard deviation of 4.91, indicating that the prediction errors are centered close to zero.

The Residuals vs Predicted plot in Figure 2 shows the distribution of residuals around the zero line, while the residual distribution histogram illustrates the overall spread of prediction errors. The Actual vs Predicted plot shows most observations located close to the diagonal reference line, indicating strong agreement between predicted and actual values [9, ch. 3].

4.5 Learning Curves

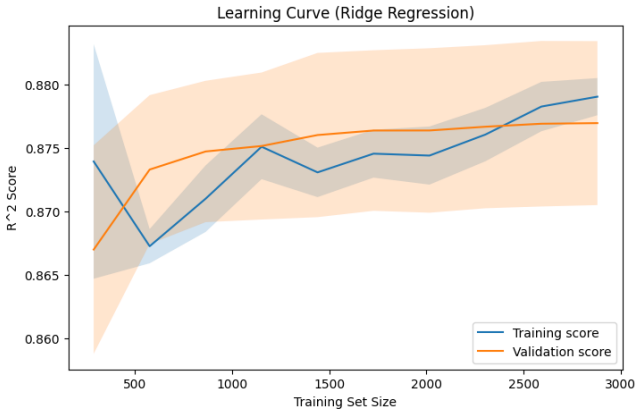

Figure 3: Learning curve of the Ridge Regression model showing the training and validation R^2 scores as the size of the training set increases.

To further examine the behavior of the best-performing model, learning curves were generated for the Ridge Regression model. Figure 3 shows the training and validation R^2 scores as the size of the training set increases. Both curves gradually converge as more training data is used, with the validation score stabilizing around $R^2 = 0.877$–0.878.

The relatively small gap between the training and validation curves indicates consistent model performance as the dataset size increases. Learning curves are commonly used to evaluate model performance and generalization behavior [7].

4.6 Ridge Coefficients

To examine the contribution of individual features in the best-performing model, the coefficients of the Ridge Regression model were extracted and ranked by absolute magnitude. Figure 4 shows the top coefficients influencing the model predictions.

The largest positive coefficients correspond to `num_tasks_automated_percent` (7.36), `num_experience_years` (5.69), and `num_focus_hours_per_day` (5.49), indicating that automation, experience, and focused work time contribute positively to the predicted productivity score. In contrast, the largest negative coefficients are `num_error_rate_percent` (-4.63) and `num_burnout_risk_score` (-4.56), suggesting that higher error rates and burnout levels reduce predicted productivity.

4.7 Final Model Evaluation (L2 Ridge)

Metric	Value
MAE	3.9428
RMSE	4.9105
R^2	0.8788

Table 4: Final model evaluation results on the test set.

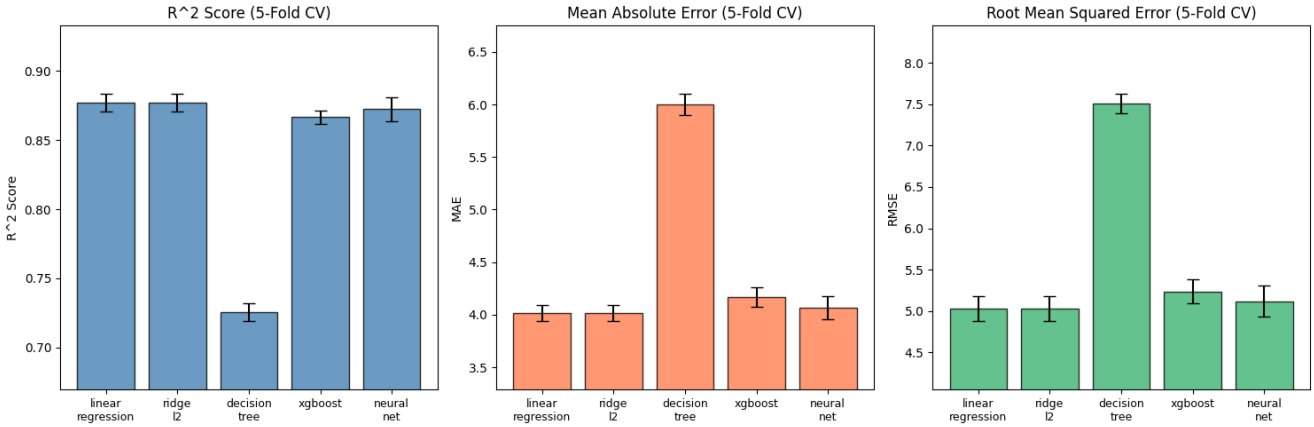

Figure 1: Model comparison using 5-fold cross-validation.

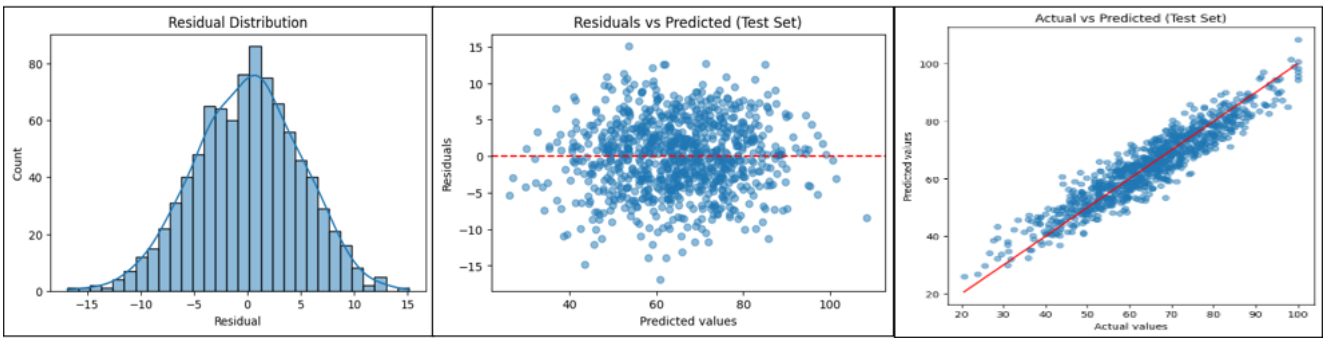

Figure 2: Residual analysis of the final Ridge Regression model on the test set, including residual distribution, residuals vs predicted values, and actual vs predicted values.

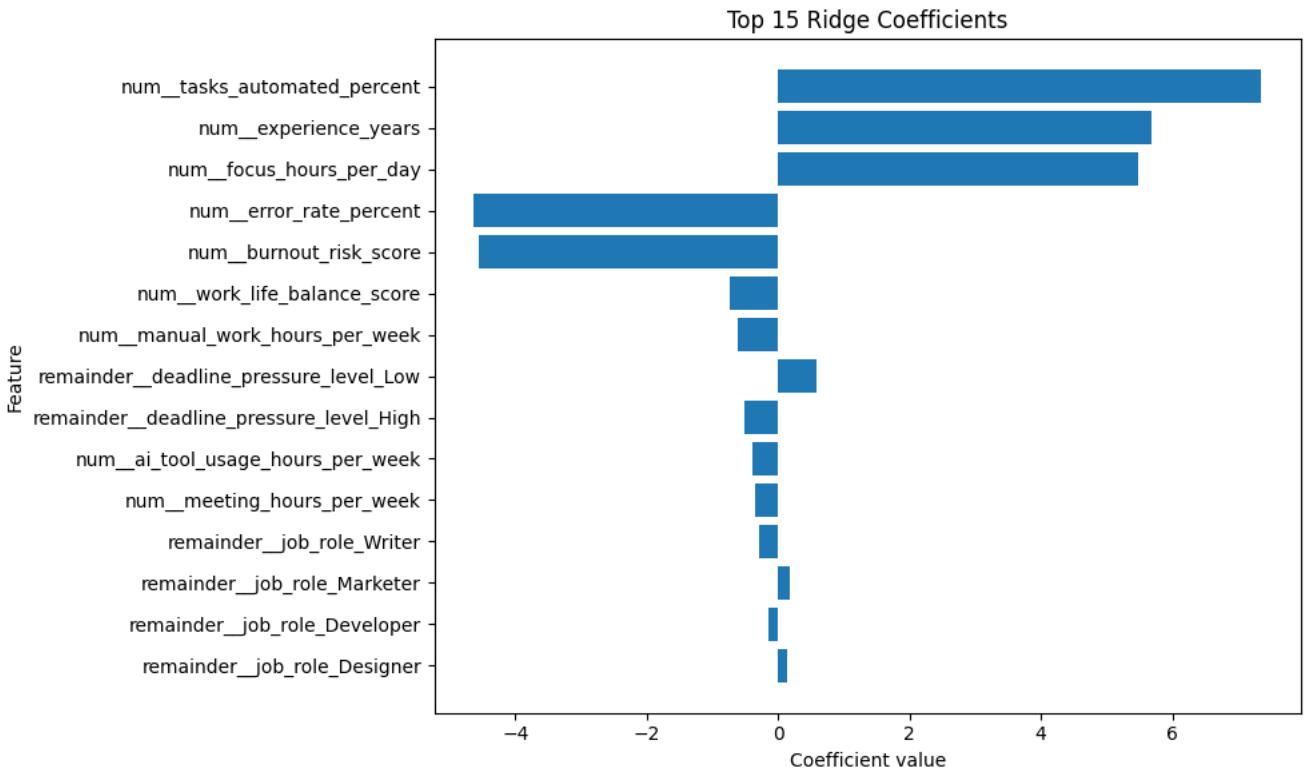

Figure 4: Top 15 Ridge Regression coefficients ranked by absolute magnitude. Positive coefficients indicate features that increase the predicted productivity score, while negative coefficients indicate features that decrease it.

Table 4 presents the final evaluation of the selected best-performing model, Ridge Regression with L2 regularization, on the held-out test set. The model achieves a Mean Absolute Error (MAE) of 3.9428, a Root Mean Squared Error (RMSE) of 4.9105, and an R^2 score of 0.8788. These results indicate that the model explains approximately 87.9% of the variance in the target variable while maintaining relatively low prediction errors [9].

5 Discussion

5.1 Cross-validation performance

The results suggest that the relationships present in the dataset are largely linear, which explains why Linear Regression and Ridge Regression outperform the more complex models. The very small difference between these two models indicates that L2 regularization provides only a marginal improvement in predictive performance, although it slightly improves the R^2 score and therefore is selected as the final model.

The weaker performance of the Decision Tree indicates that simple tree-based models struggle to capture the underlying patterns in this dataset. Although XGBoost and the Neural Network introduce greater model complexity and were tuned using simulated annealing within the cross-validation process, they do not provide a meaningful improvement over the simpler linear approaches. Overall, the findings indicate that the simpler linear models offer the most reliable and stable predictions for this task [9, ch. 6; ch. 8].

5.2 Statistical significance test

The statistical significance analysis indicates that Ridge Regression does not provide a meaningful improvement over the baseline Linear Regression model. In contrast, the Decision Tree model performs significantly worse, suggesting that this model is less suitable for capturing the relationships present in the dataset. XGBoost and the Neural Network also show statistically significant differences compared to Linear Regression, indicating slightly inferior performance in this task. Overall, the results reinforce the earlier findings that simpler linear models provide the most reliable performance for this dataset.

5.3 Model Comparison (Error Bars)

The comparison suggests that the underlying relationships in the dataset are largely linear, which explains why Linear Regression and Ridge Regression outperform more complex models. Tree-based methods such as Decision Trees appear to struggle to capture the patterns effectively, leading to larger prediction errors. The slightly lower performance of XGBoost and the Neural Network indicates that additional model complexity does not provide a clear advantage for this dataset. The relatively small error bars for the linear models also suggest stable performance across cross-validation folds, indicating good generalization.

5.4 Residual Analysis

The residual analysis suggests that the model produces stable and unbiased predictions. The residuals are centered around zero, indicating that the model does not systematically overpredict or underpredict the target variable. The random scatter of residuals in the Residuals vs Predicted plot suggests that the model captures the underlying relationships in the data without clear patterns of error. Additionally, the approximately symmetric residual distribution indicates well-behaved error terms. The strong alignment between predicted and actual values further confirms that the model achieves good predictive performance on unseen data [9, ch. 3].

5.5 Learning curves

The learning curves suggest that the Ridge Regression model achieves a good balance between bias and variance. The small difference between the training and validation scores indicates that the model does not significantly overfit the training data and generalizes well to unseen data. Additionally, the flattening of both curves at larger training sizes suggests that the model has reached a stable performance level and that adding substantially more data would likely provide only limited improvements [7, ch. 7].

5.6 Ridge Coefficients

The coefficient analysis provides insight into the factors influencing productivity in the dataset. Features related to automation, experience, and focused working time show strong positive effects, suggesting that these variables contribute significantly to improved productivity. Conversely, variables associated with higher error rates and burnout display strong negative relationships, indicating that these factors reduce performance.

These findings are consistent with the assumptions made during the preprocessing stage and support the relevance of the engineered features used in the model. Because Ridge Regression applies L2 regularization, the coefficients are shrunk toward zero, which reduces overfitting while maintaining interpretability of feature importance [7, ch. 7].

5.7 Final model evaluation (L2 Ridge)

The evaluation on the unseen test set confirms the strong predictive performance of the Ridge Regression model. The high R^2 score indicates that the model captures most of the variability in the target variable, while the relatively low MAE and RMSE values suggest accurate and stable predictions. The consistency between the cross-validation results and the final test performance further indicates that the model generalizes well to unseen data [9, ch. 6].

6 Limitations and Future Work

Several practical limitations were encountered during the experiments. The models were tested on different hardware configurations (Apple M1, Intel i5 11th gen-

eration, and Intel i7 13th generation), and the final experiments were conducted on the Intel i7 13th generation system, which provided the best performance. Even with this hardware, the full run required approximately 263 minutes to complete. The relatively long execution time is mainly due to the use of 5-fold cross-validation, the evaluation of multiple models, and the simulated annealing hyperparameter search, which repeatedly evaluates different parameter configurations.

Another issue encountered during development was that some models initially used fixed hyperparameters, which limited their performance. To address this, a simulated annealing search procedure was implemented to obtain better hyperparameter configurations. Additionally, cross-validation was chosen instead of a single train-validation split because it provides more reliable and stable performance estimates across different data partitions.

For future work, more advanced hyperparameter optimization techniques and larger search spaces could be explored to further improve model performance. Additionally, improvements in computational efficiency, such as parallelization or GPU acceleration, could significantly reduce training time.

7 Conclusion

From the experimental results we can answer the research question and the two secondary questions of this study.

RQ Answer: The results show that simpler linear models perform best for predicting worker productivity in this dataset. Ridge Regression achieved the highest performance with an R^2 score of 0.8788 on the test set and the lowest prediction errors. Decision Trees performed significantly worse, while XGBoost and the Neural Network achieved slightly lower performance than the linear models. Overall, the results suggest that the relationships in the dataset are largely linear and can be effectively captured using regularized linear regression.

SQ1 Answer: The most predictive features are related to automation, experience, and focused working time. In particular, `num_tasks_automated_percent`, `num_experience_years`, and `num_focus_hours_per_day` show the strongest positive effects on predicted productivity. Conversely, `num_error_rate_percent` and `num_burnout_risk_score` have strong negative coefficients, indicating that higher error rates and burnout reduce productivity.

SQ2 Answer: Hyperparameter selection through simulated annealing improved the performance and stability of the models by identifying suitable parameter configurations. For example, the best Ridge model used a regularization parameter of $\alpha = 4.1785$, while the Decision Tree achieved its best performance with a maximum depth of 10 and a minimum leaf size of 11. XGBoost performed best with 821 trees and a learning rate of 0.0168, while the Neural Network achieved its best results with a two-layer architecture of (22, 12) neurons and $\alpha = 0.084$. Despite these optimizations, more complex models still did not outperform the simpler Ridge Regression model, indicating that the structure of the dataset favors linear approaches.

References

[1] Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. Generative ai at work. The Quarterly Journal of Economics, 2025. doi:10.1093/qje/qjae044.

[2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. URL: https://arxiv.org/pdf/1603.02754, doi:10.1145/2939672.2939785.

[3] Fabrizio Dell'Acqua, Edward McFowland III, Ethan Mollick, Hila Lifshitz-Assaf, Katherine C. Kellogg, Saran Rajendran, Lisa Krayer, François Candelon, and Karim R. Lakhani. Navigating the jagged technological frontier: Field experimental evidence of the effects of ai on knowledge worker productivity and quality. Organization Science, 2026. doi:10.1287/orsc.2025.21838.

[4] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation, 1998. doi:10.1162/089976698300017197.

[5] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011. URL: https://proceedings.mlr.press/v15/glorot11a/glorot11a.pdf.

[6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep Learning. MIT Press, 2016. URL: http://www.deeplearningbook.org.

[7] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer, 2009. URL: https://hastie.su.domains/ElemStatLearn/, doi:10.1007/978-0-387-84858-7.

[8] Timothy O. Hodson. Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. Geoscientific Model Development, 2022. doi:10.5194/gmd-15-5481-2022.

[9] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. An Introduction to Statistical Learning with Applications in R. Springer, 2 edition, 2021. URL: https://www.statlearning.com/, doi:10.1007/978-1-0716-1418-1.

[10] Timothy A. Judge, Carl J. Thoresen, Joyce E. Bono, and Gregory K. Patton. The job satisfaction–job performance relationship: A qualitative and quantitative review. Psychological Bulletin, 2001. doi:10.1037/0033-2909.127.3.376.

[11] Vishard Mehta. Ai workplace productivity dataset, 2024. Kaggle dataset. URL: https://www.kaggle.com/datasets/vishardmehta/ai-tool-usage-and-workplace-productivity-dataset/data.

[12] Shakked Noy and Whitney Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. Science, 2023. doi:10.1126/science.adh2586.

[13] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of ai on developer productivity: Evidence from github copilot. arXiv preprint arXiv:2302.06590, 2023. URL: https://arxiv.org/abs/2302.06590, doi:10.48550/arXiv.2302.06590.

MLVU information sheet

Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.

Group number
157

Authors

name	student number
Zoe Lea Petroianu	(2809310)
Aida Marija Bielinskaite	(2804099)
George Dedempilis	(2803764)

Software used *Describe briefly which software and libraries you relied on. If you built a particular algorithm from scratch, make sure to state it explicitly here.*

All experiments were conducted in Python 3 using Jupyter notebooks. Data preprocessing and manipulation were handled using NumPy and pandas. All models were implemented in Python using scikit-learn, except for XGBoost, which was implemented with the XGBoost library. The neural network was implemented using scikit-learn's MLPRegressor. Hyperparameter optimization was performed using the `dual_annealing` function from SciPy. Model evaluation and visualization were carried out using Matplotlib and Seaborn. Statistical comparison of model performance across folds was conducted using a paired t-test via SciPy's `ttest_rel` function. Learning curves were generated using scikit-learn's `learning_curve` function.

Use of AI tools *If you used tools like ChatGPT or Github Copilot in any phase of the report, please detail here which tools you used, what parts of the work you used them for and how. Note that if you use an AI tool and don't report it, it's considered fraud, so be detailed. Note also that having an AI tool write for you is not allowed. If no AI tools were used, please state that explicitly.*

We used ChatGPT in a limited supporting role when preparing this report. Specifically, we used it for brainstorming, generating ideas about possible directions for the report, getting suggestions of relevant resources, checking to confirm whether our structure aligned with the course rubric, and checking for language/grammar mistakes. The resources suggested by the tool were checked by us before being used.

The written content of the report, as well as the code, were produced by us.

Link to code (optional) *If you would like to share your code, you can link to your repository here.*

Group disagreements (optional) *If there are any disagreements in the group, you may **not** remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here*